

Enhancing 6D Object Pose Estimation

Mohammad Fakih

s348034@studenti.polito.it

Ulugbek Rakhmatullaev

s330568@studenti.polito.it

Karim Abdelgelil Mohamed Shalaby

s349265@studenti.polito.it

Abstract

*Estimating the 6D pose of objects from RGB images is a challenging problem due to sensitivity to lighting conditions, textureless surfaces, and occlusions. In order to overcome these constraints, we suggest a two-phase framework that uses RGB input to estimate object pose before expanding the model to use depth information for increased robustness and accuracy. RGBD based methods have shown strong performance by using both color and depth information, but their effectiveness relies on how well these two complementary modalities are combined. In this two stage framework, we evaluated the two models on the LineMOD dataset. We first introduce a pure RGB baseline, where objects are localized using a YOLOv8 detector, and a ResNet50 network predicts object rotation, whereas a small CNN network predicts depth, while the remaining translation components are recovered analytically using the pinhole camera model. Based on this baseline, we extend our framework to an RGBD model by using depth data to improve geometric understanding and pose accuracy. Results indicate that the proposed RGB-based method provides a good monocular baseline, and that extending the framework to RGBD provides notable gains in pose accuracy and robustness due to the availability of explicit geometric information.*¹

1. Introduction

Estimating the 6D pose of rigid objects from visual observations is fundamental to robotics and computer vision, with applications in robotic grasping, augmented reality, and autonomous manipulation. The task requires recovering 3D rotation and translation relative to the camera, which remains challenging due to occlusions, textureless surfaces, object symmetries, and the inherent ambiguity of monocular depth perception.

Recent learning-based methods have made significant progress on this problem. PoseCNN [7] pioneered end-to-end deep learning for pose estimation, directly regressing pose from CNN features. DenseFusion [6] achieved strong results by combining RGB and depth through dense per-pixel feature fusion with iterative refinement. However, these methods either require complex multi-stage pipelines or expensive iterative inference.

We present a simpler two-stage framework for 6D pose estimation on LineMOD, comparing RGB-only and RGB-D approaches. Both use YOLOv8 for detection and segmentation, followed by a pose network. Our key contribution is *geometric translation*: instead of regressing all translation components directly, we predict only Z-depth and derive X, Y analytically using the pinhole camera model. This explicitly embeds physical constraints into the learning pipeline, reducing the regression burden to rotation and depth estimation only.

For the RGB-D model, we further exploit depth sensor readings through a z-offset formulation, predicting small surface-to-center corrections (± 10 -30mm) rather than absolute depth (0.5-1.5m). This allows the network to leverage the sensor measurement as a strong geometric prior. We train both models using homoscedastic uncertainty weighting [4] to automatically balance rotation and translation losses.

Results show our RGB model achieves 41.2% ADD-2cm accuracy, while the RGB-D model reaches 84.9% (a $2\times$ improvement). While our method does not match state-of-the-art iterative approaches on all objects, it excels on compact objects and offers simpler single-pass inference.

2. Related Work

6D object pose estimation has been extensively studied in computer vision. We review key approaches and position our work relative to the existing literature.

Template-based Methods. Early methods relied on matching observed image features to pre-computed tem-

¹Code and pretrained models: <https://github.com/SFR-Vision/6d-pose-estimation>

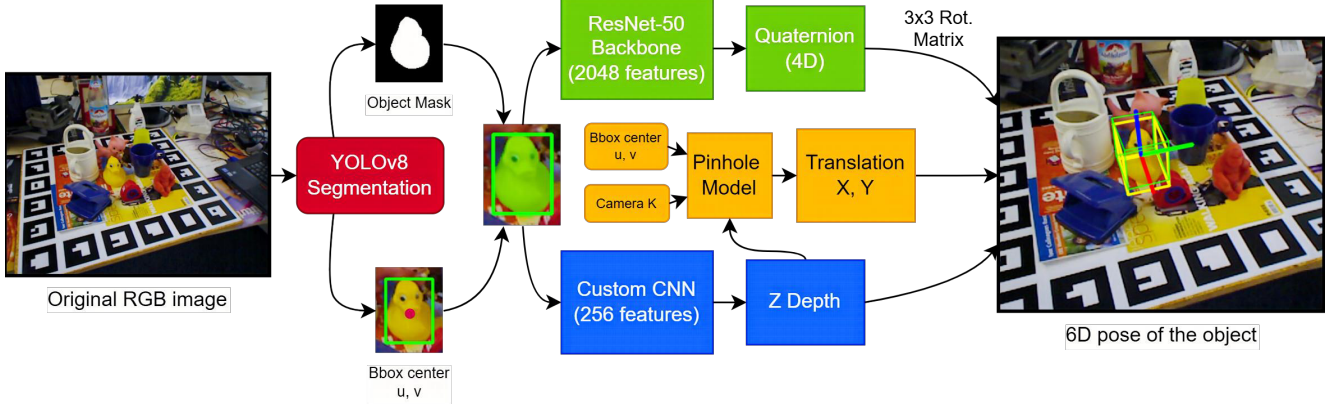


Figure 1. RGB model pipeline.

plates of objects at various viewpoints. LineMOD [2] introduced a multimodal template matching approach using color gradients and surface normals, establishing the benchmark dataset we use for evaluation. While effective for texture-rich objects, these methods struggle with occlusion and require extensive template databases.

Keypoint-based Methods. Several approaches frame pose estimation as 2D keypoint detection followed by Perspective-n-Point (PnP) solving. These methods predict heatmaps for distinctive 3D model keypoints projected onto the image, then recover pose through geometric optimization. While providing strong geometric consistency, keypoint-based methods can fail on textureless objects or under heavy occlusion.

Direct Regression Methods. PoseCNN [7] pioneered end-to-end deep learning for pose estimation, directly regressing object center, depth, and rotation from CNN features. This approach introduced the ADD metric that has become the standard for evaluation. We follow this direct regression paradigm but incorporate explicit geometric constraints for translation prediction rather than regressing all six degrees of freedom independently.

RGB-D Fusion Methods. DenseFusion [6] proposed dense per-pixel feature fusion of RGB and depth modalities, achieving state-of-the-art results through iterative pose refinement. Our RGB-D model similarly combines appearance and depth cues, but employs a simpler dual-stream architecture that fuses features at the prediction stage. Rather than iterative refinement, we leverage depth as a geometric prior for the Z-translation component.

3. Methodology

Overview. Our pipeline consists of two stages: (1) object detection and segmentation using YOLOv8, and (2) pose estimation using either an RGB-only or RGB-D model. Both models predict rotation as a quaternion and leverage geometric constraints from the pinhole camera model to derive translation.

Object Detection. We use YOLOv8n-seg [3] fine-tuned on LineMOD, which outputs bounding boxes, instance segmentation masks, and class labels for each detected object.

3.1. RGB Model

The RGB model pipeline is shown in Fig. 1. From each detection, we extract a 224×224 crop with $1.2 \times$ padding, concatenated with the binary mask to form a 4-channel input. RGB channels are normalized using ImageNet statistics.

3.1.1. Two-Stream Architecture

Table 1. RGB Model Architecture (26.6M params)*

Layer	Config	Output
<i>Rotation Stream (ResNet-50)</i>		
Conv1 Layer1-4	$4 \rightarrow 64, 7 \times 7, s2$ Standard ResNet-50	64×112 2048
<i>Rotation Head</i>		
FC	$2048 \rightarrow 1024 \rightarrow 512 \rightarrow 4, L2\text{-norm}$	4
<i>Depth Stream (CNN)</i>		
Conv1-4	$4 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$	256
<i>Z-Depth Head</i>		
FC	$256 \rightarrow 128 \rightarrow 64 \rightarrow 1, \text{Sigmoid}$	1

* All FC layers include BatchNorm, ReLU, and Dropout between them.

The model employs two parallel streams (Tab. 1):

Rotation Stream: A modified ResNet-50 [1] backbone with the first conv layer adapted to accept 4 chan-

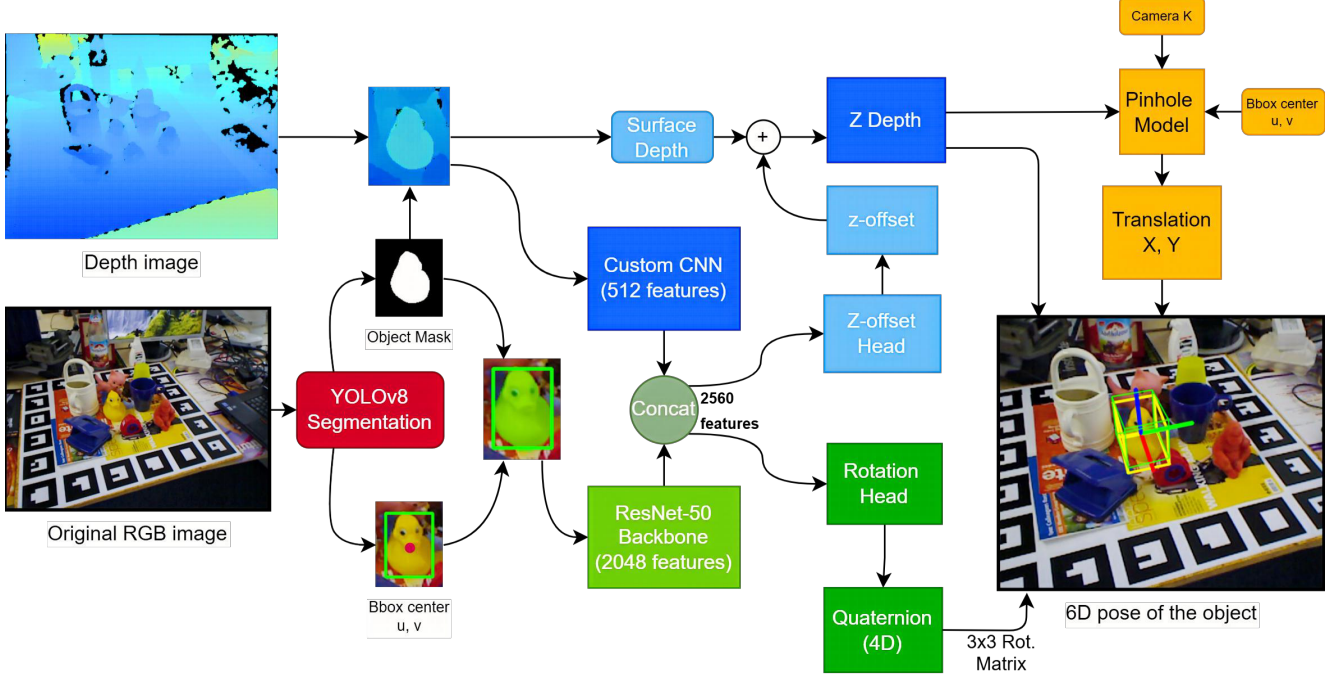


Figure 2. RGB-D model architecture.

nels (RGB weights from ImageNet, mask channel initialized with small random values). The backbone outputs 2048-D features feeding a rotation head that produces a unit quaternion.

Depth Stream: A lightweight CNN produces 256-D features, feeding a depth head that outputs Z-depth scaled to $[0.3, 1.7]$ m (determined by analyzing LineMOD ground truth depths: 0.51-1.45m).

3.1.2. Geometric Translation

X and Y translation are derived analytically using the pin-hole camera model:

$$t_x = \frac{(u - c_x) \cdot t_z}{f_x}, \quad t_y = \frac{(v - c_y) \cdot t_z}{f_y} \quad (1)$$

where (u, v) is the bounding box center, t_z is predicted depth, and (f_x, f_y, c_x, c_y) are camera intrinsics. This reduces the learning problem to rotation and depth only.

3.2. RGB-D Model

The RGB-D model (Fig. 2) extends the RGB model with depth information, forming a 5-channel input (RGB + Depth + Mask). The depth channel is preprocessed with bilateral filtering and normalized to $[0, 1]$.

3.2.1. Dual-Stream Fusion

The 5-channel input is split into two streams (Tab. 2):

RGB+Mask Stream (4ch): ResNet-50 backbone producing 2048-D features.

Depth+Mask Stream (2ch): Custom CNN producing 512-D features.

Features are concatenated (2560-D) and fed to rotation and z-offset heads.

Table 2. RGB-D Model Architecture (29.7M params)*

Layer	Config	Output
<i>RGB+Mask Stream (ResNet-50)</i>		
Conv1, Layer1-4	Same as RGB model	2048
<i>Depth+Mask Stream (CNN)</i>		
Conv1-4	2→64→128→256→512	512
<i>Feature Fusion + Heads</i>		
Concat	[RGB; Depth]	2560
Rotation Head	2560→1024→512→4	4
Z-Offset Head	2560→512→128→1	1

* All FC layers include BatchNorm, ReLU, and Dropout between them.

3.2.2. Z-Offset Formulation

Instead of predicting absolute depth, the RGB-D model predicts a small offset from the sensor measurement: $z_{\text{final}} = z_{\text{sensor}} + z_{\text{offset}}$. The sensor depth (median of masked pixels) represents the visible surface; the network learns to adjust this to the object center (typically ± 20 -50mm).

Benefits: (1) Simpler learning target (small corrections vs absolute depth), (2) leverages accurate sensor measurements as prior, (3) better initialization ($z_{\text{offset}}=0$ gives reasonable estimate).

X and Y translations use the same geometric formulation as the RGB model.

4. Dataset

LineMOD Benchmark. We evaluate our approach on the LineMOD dataset [2], a standard benchmark for 6D object pose estimation. The dataset contains 13 household objects with ground truth 6D poses and segmentation masks. Objects exhibit diverse geometric properties including varying sizes, symmetries, and surface characteristics.

Objects. The 13 objects (Fig. 5) in the dataset are:

- **Asymmetric objects (11):** ape, benchvise, camera, can, cat, drill, duck, holepuncher, iron, lamp, phone
- **Symmetric objects (2):** eggbox, glue (rotational symmetry)

For symmetric objects, we use the ADD-S metric instead of ADD during evaluation to account for pose ambiguity.

Data Statistics. We use the preprocessed version of LineMOD with the following split:

Split	Images
Training	12,637 (80%)
Validation	1,573 (10%)
Test	1,573 (10%)
Total	15,783

Camera Intrinsics. All images are captured with fixed camera parameters:

Parameter	Value
f_x (focal length x)	572.4 px
f_y (focal length y)	573.6 px
c_x (principal point x)	325.3 px
c_y (principal point y)	242.0 px
Image resolution	640×480

Depth Statistics. Analysis of the ground truth annotations reveals that object depths (Z-coordinate of object center) range from 0.51m to 1.45m. We use this information to optimize our depth prediction range to $[0.3, 1.7]$ m, providing sufficient margin while focusing network capacity on the relevant depth range.

Data Preprocessing. For each training sample, we perform the following preprocessing:

1. **Crop extraction:** A square crop centered on the ground truth bounding box with $1.2 \times$ padding is extracted from the full image.
2. **Resizing:** The crop is resized to 224×224 pixels.

3. **RGB normalization:** RGB channels are normalized using ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).
4. **Mask binarization:** Segmentation masks are converted to binary (0 or 1).
5. **Depth preprocessing (RGB-D only):** Bilateral filtering is applied to reduce depth noise while preserving edges, followed by normalization to $[0, 1]$ based on the range $[0.3, 1.7]$ m.

5. Training and Testing

5.1. Training

Training Loss Functions. Pose estimation is inherently a multi-task learning problem: rotation (in radians) and translation (in meters) have different units and magnitudes. Naively weighting these losses equally often leads to sub-optimal convergence.

Auto-Weighted Multi-Task Loss. We adopt homoscedastic uncertainty weighting [4] to automatically learn task-dependent weights:

$$\mathcal{L} = \frac{1}{2}e^{-s_r}\mathcal{L}_{rot} + \frac{1}{2}e^{-s_t}\mathcal{L}_{trans} + \frac{1}{2}(s_r + s_t) \quad (2)$$

where $s_r = \log(\sigma_r^2)$ and $s_t = \log(\sigma_t^2)$ are learned uncertainty parameters, initialized to 0 (i.e., $\sigma^2 = 1$). This creates a self-balancing mechanism: when a task’s loss is high, the network can reduce its contribution by increasing σ^2 ; however, the regularization term $\frac{1}{2}(s_r + s_t)$ prevents the network from ignoring difficult tasks.

Why Losses Become Negative. A notable characteristic of this formulation is that the total loss can become negative during training. This occurs because the regularization term $\frac{1}{2}(s_r + s_t)$ can be negative when $s_r, s_t < 0$ (i.e., when $\sigma^2 < 1$). As training progresses and the individual losses $\mathcal{L}_{rot}$ and $\mathcal{L}_{trans}$ decrease, the network learns smaller uncertainty values, causing the regularization term to become increasingly negative. This is expected and correct behavior, the loss magnitude is not meaningful in absolute terms, but the gradients remain valid for optimization. What matters is that the loss decreases monotonically, indicating learning progress.

Rotation Loss. We use a quaternion angle formulation with atan2 for numerical stability:

$$\mathcal{L}_{rot} = 2 \cdot \text{atan2}(\|\mathbf{q}_{pred} - \mathbf{q}_{gt}\|_2, \|\mathbf{q}_{pred} + \mathbf{q}_{gt}\|_2) \quad (3)$$

This computes the geodesic distance on the quaternion sphere. To handle the quaternion double-cover ($\mathbf{q}$ and $-\mathbf{q}$ represent the same rotation), we flip $\mathbf{q}_{gt}$ if $\mathbf{q}_{pred} \cdot \mathbf{q}_{gt} < 0$.

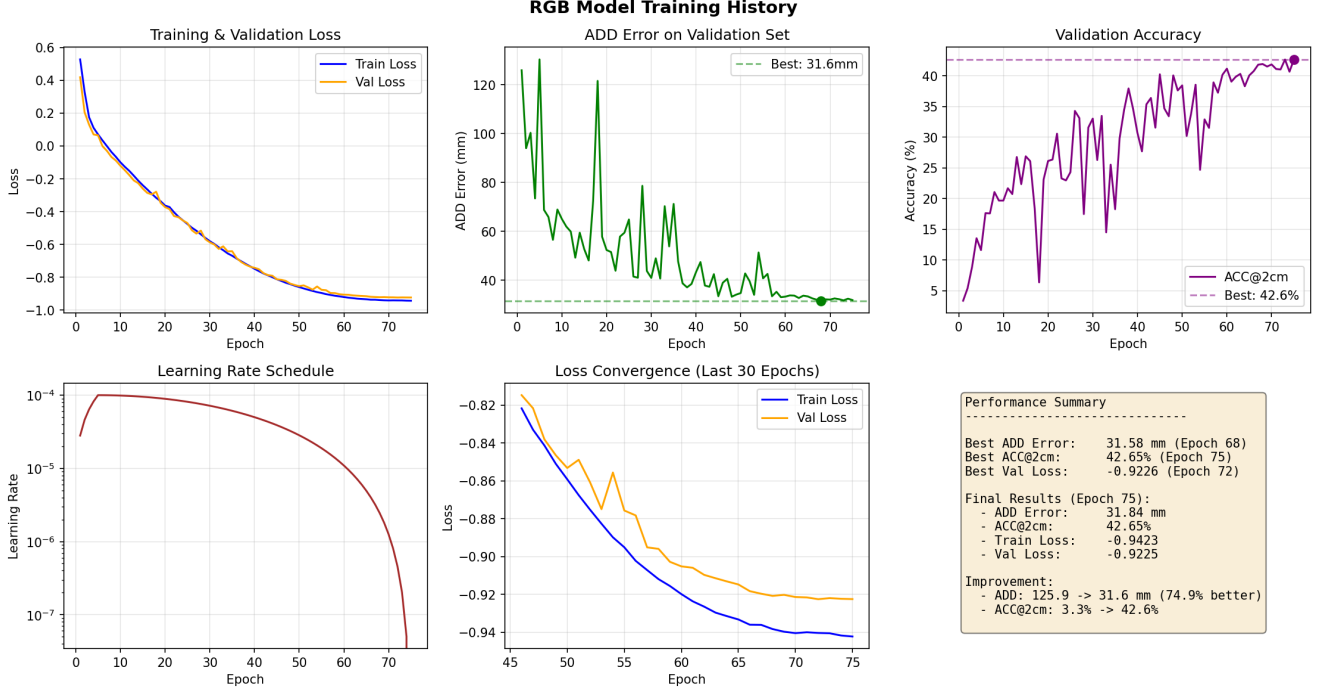


Figure 3. RGB model training curves showing loss convergence, ADD error, accuracy, and learning rate schedule.

Translation Loss. We supervise only Z-depth with L1 loss (X, Y are derived geometrically):

$$\mathcal{L}_{trans} = |t_{z,pred} - t_{z,gt}| \quad (4)$$

L1 loss is more robust to outliers than L2. For the RGB-D model, we supervise the final depth $t_z = z_{sensor} + \Delta z_{offset}$, not the offset directly.

Optimization. We use AdamW [5] optimizer, which decouples weight decay from the gradient update. Unlike standard Adam with L2 regularization, AdamW applies weight decay directly to the weights, providing more effective regularization for transfer learning from pretrained backbones. This is particularly important for our modified ResNet-50 where we want to regularize all weights uniformly while preserving the pretrained feature extractors.

Hyperparameter	Value
Optimizer	AdamW
Learning rate	1×10^{-4}
Weight decay	1×10^{-4}
Batch size	48
Epochs	75
Gradient clipping	max_norm=1.0

Learning Rate Schedule. We employ a two-phase schedule:

1. **Warmup (epochs 1-5):** Linear warmup from 10% to full learning rate.
2. **Decay (epochs 6-75):** Cosine annealing to near zero.

YOLOv8 Training. YOLOv8n-seg (nano variant, pre-trained on COCO) is fine-tuned on LineMOD for object detection and instance segmentation:

Parameter	Value
Base model	YOLOv8n-seg (COCO)
Epochs	50
Image size	640×640 (from 640×480)
Batch size	48
Optimizer	AdamW (lr=0.001)
Early stopping	patience=20

The fine-tuned model achieves >95% mAP50 on LineMOD test set, providing reliable detection and segmentation masks for the pose estimation stage. Training takes approximately 1 hour on a single GPU.

Training Curves. Fig. 3 shows the RGB model training history. The model converges to 42.6% validation ACC@2cm with 31.6mm ADD error (final test set: 41.2%, 31.5mm).

Fig. 4 shows the RGBD model training history. The model achieves 84.5% validation ACC@2cm with 13.9mm

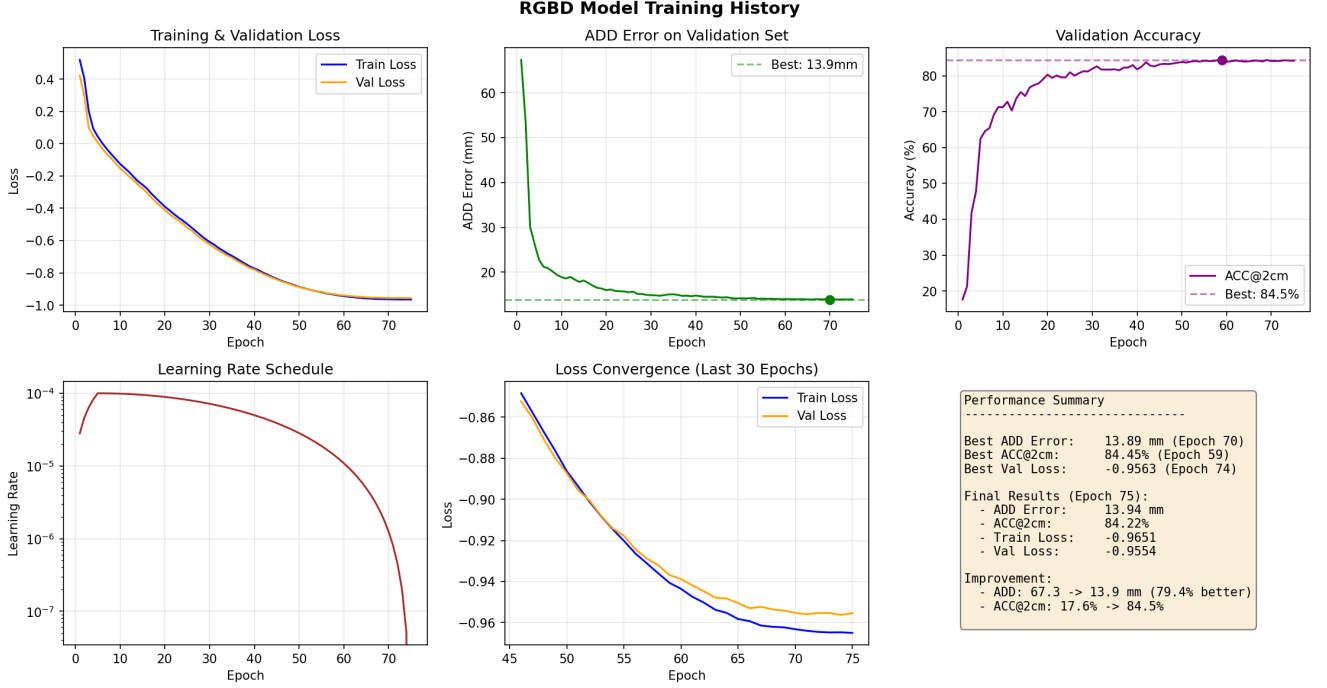


Figure 4. RGB-D model training curves. The depth information enables faster convergence and significantly better final accuracy.

ADD error (final test set: 84.9% ACC, 13.5mm ADD)—a significant improvement over the RGB model due to the depth prior.

5.2. Testing

Evaluation Loss (ADD Metric). Unlike training losses which measure rotation and translation separately, evaluation uses the ADD (Average Distance of Model Points) metric which measures the actual 3D pose error:

$$\text{ADD} = \frac{1}{m} \sum_{i=1}^m \|(\mathbf{R}_{pred}\mathbf{p}_i + \mathbf{t}_{pred}) - (\mathbf{R}_{gt}\mathbf{p}_i + \mathbf{t}_{gt})\|_2 \quad (5)$$

where $\{\mathbf{p}_i\}_{i=1}^m$ are $m = 500$ downsampled 3D model points. ADD is also computed during training on the validation set for monitoring purposes, separate from the training loss used for backpropagation.

ADD-S (for Symmetric Objects). For objects with rotational symmetry (eggbox, glue), we use closest point matching:

$$\text{ADD-S} = \frac{1}{m} \sum_{i=1}^m \min_j \|(\mathbf{R}_{pred}\mathbf{p}_i + \mathbf{t}_{pred}) - (\mathbf{R}_{gt}\mathbf{p}_j + \mathbf{t}_{gt})\|_2 \quad (6)$$

ADD-2cm Accuracy. A pose is considered correct if ADD (or ADD-S) is less than 20mm.

Inference Pipeline. During testing:

1. YOLOv8 detects and segments objects in the input image
2. For each detection, extract crop and run pose model
3. Compute ADD/ADD-S against ground truth

6. Experiments

6.1. Main Results

Tab. 3 shows our main results on LineMOD.

Table 3. Comparison of RGB and RGB-D models on LineMOD test set.

Model	Mean ADD (mm) ↓	ADD-2cm (%) ↑
RGB	31.5	41.2
RGBD	13.5	84.9
Improvement	-57.1%	+106.1%

The RGB-D model achieves 57% lower ADD error and $2\times$ higher accuracy by leveraging depth information. The z-offset formulation allows the network to learn only small corrections to the sensor depth.

Comparison with State-of-the-Art. Tab. 4 compares our methods with published results on LineMOD.

Table 4. Comparison with state-of-the-art methods (ADD accuracy %).

Object	RGB		RGB-D			
	PoseCNN [7]	Ours	PointFusion [8]	DenseFusion [6]	DF (iter.)	Ours
ape	77.0	38.2	70.4	79.5	92.3	98.4
benchvise	97.5	29.8	80.7	84.2	93.2	68.6
camera	93.5	34.2	60.8	76.5	94.4	79.2
can	96.5	41.2	61.1	86.6	93.1	89.9
cat	82.1	48.7	79.1	88.8	96.5	100.0
driller	95.0	31.4	47.3	77.7	87.0	68.6
duck	77.7	48.8	63.0	76.3	92.3	98.4
eggbox	97.1	72.8	99.9	99.9	99.8	100.0
glue	99.4	73.0	99.3	99.4	100.0	100.0
holepuncher	52.8	36.6	71.8	79.0	92.1	94.3
iron	98.3	19.1	83.2	92.1	97.0	49.6
lamp	97.5	32.8	62.3	92.3	95.3	86.1
phone	87.7	29.5	78.8	88.0	92.8	70.5
MEAN	88.6	41.2	73.7	86.2	94.3	84.9

Analysis: Our RGB model (41.2%) underperforms PoseCNN+DeepIM (88.6%), which uses iterative refinement and additional training data. Our RGB-D model (84.9%) approaches DenseFusion per-pixel (86.2%) without iterative refinement, but falls short of DenseFusion iterative (94.3%). Key differences include:

- PoseCNN uses iterative pose refinement with DeepIM
- DenseFusion employs dense per-pixel fusion and iterative refinement
- Our approach prioritizes simplicity: single-pass inference with geometric translation

Per-Object Analysis. Based on the comparison in Tab. 4, we analyze per-object performance:

Objects where our RGBD outperforms state-of-the-art:

- **ape** (98.4%): Beats DenseFusion iterative (92.3%) by 6.1%
- **cat** (100%): Outperforms all methods including DF iterative (96.5%)
- **duck** (98.4%): Strong performance on small compact object
- **eggbox** (100%): Matches best; symmetric object benefits from ADD-S
- **glue** (100%): Matches DF iterative; symmetric object
- **holepuncher** (94.3%): Outperforms DF iterative (92.1%)

Objects where our RGBD underperforms:

- **iron** (48.7% vs 97.0%): Highly reflective surface degrades our depth processing
- **benchvise** (68.6% vs 93.2%): Large object with complex geometry
- **driller** (68.6% vs 87.0%): Similar challenges to

benchvise

- **phone** (70.5% vs 92.8%): Textureless surface affects appearance features

Key insight: Our method excels on small, compact objects (ape, cat, duck) where the z-offset formulation accurately captures surface-to-center distance. It struggles with large objects (benchvise, driller) and reflective surfaces (iron) where depth sensor noise propagates through our geometric translation.

Learned Loss Weights. Both models are trained with homoscedastic uncertainty weighting. The learned weights at convergence:

Table 5. Learned loss weights at convergence.

Model	w_{rot}	w_{trans}
RGB	1.2–1.8	0.8–1.2
RGBD	1.5–2.0	0.6–0.8

The network learns to weight rotation more heavily, as translation Z-depth benefits from geometric constraints (RGB) or sensor prior (RGB-D).

6.2. Ablation Studies

We conducted several experiments to understand the impact of different training configurations.

Batch Size. Batch size 48 provided the best trade-off between gradient stability and training speed. Smaller batches (16) led to noisy gradients, while larger batches (64) showed slightly worse generalization.

Table 6. Effect of batch size on RGB model performance.

Batch Size	ADD (mm) ↓	ACC@2cm (%) ↑	Training Time
16	33.2	38.5	2.1h
32	32.1	40.8	1.8h
48	31.5	41.2	1.5h
64	31.8	40.9	1.4h

Learning Rate Schedule. We compared different learning rate strategies:

Table 7. Effect of learning rate schedule.

Schedule	ADD (mm) ↓	ACC@2cm (%) ↑
ReduceLROnPlateau	33.8	38.1
Warmup + Cosine	31.5	41.2

Why ReduceLROnPlateau underperformed: ReduceLROnPlateau waits for validation loss to plateau before reducing the learning rate, which can cause delayed reactions in training dynamics. For pose estimation with auto-weighted multi-task loss, the validation loss behavior is complex, the learned uncertainty parameters (σ_r , σ_t) can mask plateaus, causing the scheduler to trigger too late. In contrast, the predetermined warmup + cosine schedule provides consistent, predictable decay that is robust to the multi-task loss dynamics.

Data Augmentation. We tested various augmentation strategies:

Table 8. Effect of data augmentation.

Augmentation	ADD (mm) ↓	ACC@2cm (%) ↑
None (baseline)	31.5	41.2
Color jitter	35.6	32.3
Bounding box jitter	39.0	30.6

Why augmentation decreased accuracy:

- **Color jitter:** While typically beneficial for classification, pose estimation requires precise correspondence between visual appearance and 3D geometry. Color augmentation can disrupt learned texture-to-depth mappings, harming the RGB model’s ability to estimate Z-depth.
- **Bounding box jitter:** Our geometric translation relies on the bounding box center (u, v) from YOLO detection. Jittering the training crops introduces inconsistency between the crop center and the ground truth translation, corrupting the geometric relationship X,Y depend on.

This highlights a key difference from classification: pose estimation’s geometric constraints make it sensitive to spatial perturbations that classification networks easily tolerate.

Depth Range Optimization. For the RGB model, we experimented with different Z-depth prediction ranges:

Table 9. Effect of depth range on RGB model Z-prediction.

Depth Range	ADD (mm) ↓	ACC@2cm (%) ↑
[0, 3]m	34.1	38.2
[0.5, 1.5]m	41.6	29.5
[0.3, 1.7]m	31.5	41.2

Analyzing ground truth depths (0.51–1.45m), we optimized the prediction range to [0.3, 1.7]m, providing sufficient margin while focusing network capacity on the relevant depth interval.

6.3. Discussion

RGB vs RGB-D Gap: The 43% accuracy gap between RGB (41.2%) and RGB-D (84.9%) highlights the fundamental difficulty of monocular depth estimation. The RGB model must infer depth purely from appearance, while the RGB-D model benefits from direct geometric measurements.

Challenging Objects: Iron remains the most difficult object (48.7% accuracy) due to its reflective surface that degrades both appearance features and depth sensor readings. Benchvise, drill, and phone also show lower accuracy (68–70%), likely due to their larger size and complex geometry requiring more precise depth estimation.

Symmetric Objects: The ADD-S metric appropriately handles eggbox and glue, which achieve 100% accuracy with RGBD. Without ADD-S, these objects would show artificially high errors due to equivalent rotations being penalized.

Practical Implications: Our results suggest that for applications requiring more than 80% accuracy, depth sensors are essential. However, the RGB model (41.2%) may suffice for coarse manipulation tasks or as a fallback when depth is unavailable.

7. Conclusion

We presented a geometric-aware approach for 6D object pose estimation that incorporates camera intrinsics and detection geometry to improve translation prediction. Our key contributions are:

RGB Model (4-channel). Using RGB+Mask input with ResNet-50 backbone, predicting rotation as quaternion and Z-depth only. X and Y are derived geometrically from the pinhole camera model. This achieves 41.2% ADD-2cm accuracy with 31.5mm mean ADD error.

RGB-D Model (5-channel). A dual-stream fusion architecture that leverages depth sensor readings as a prior. The model predicts a small Z-offset from the sensor depth,

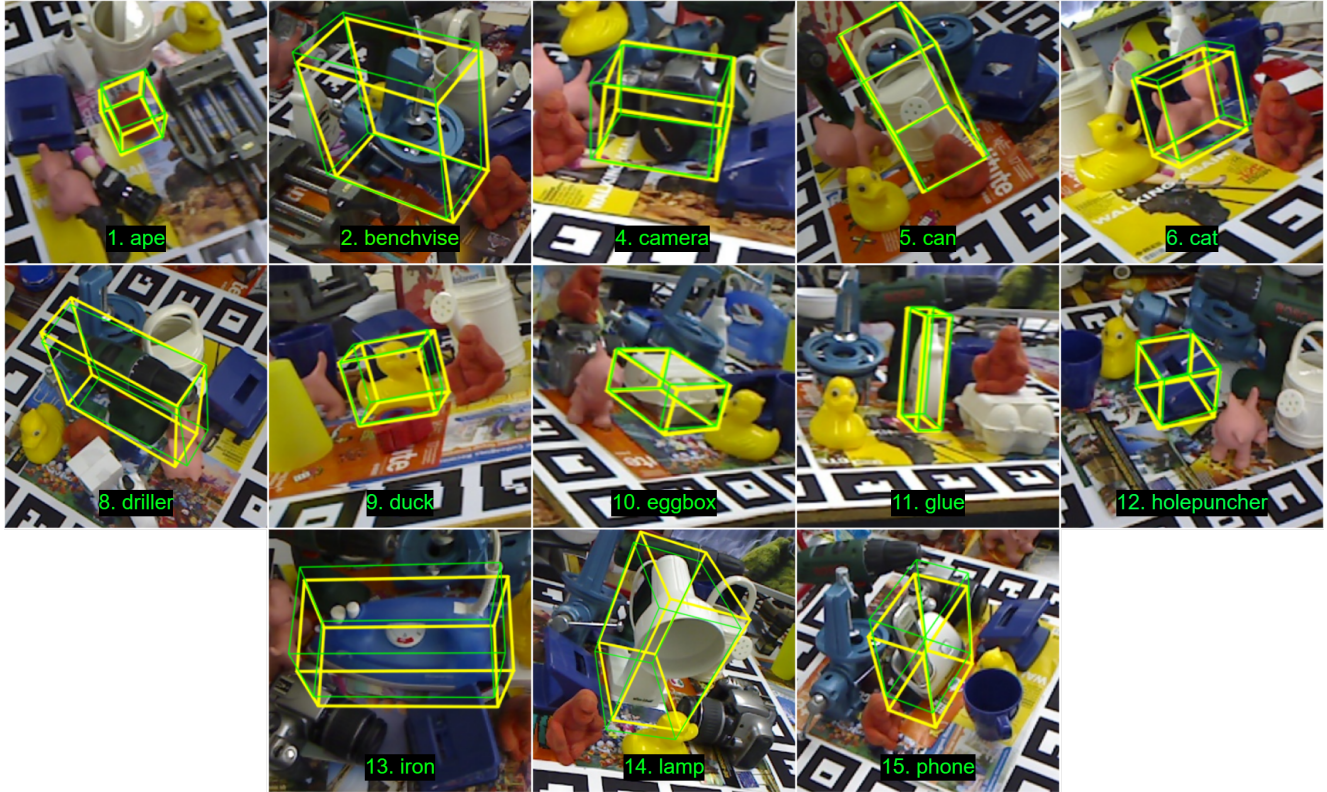


Figure 5. Qualitative results showing predicted 3D bounding boxes (yellow) vs ground truth (green) for all 13 LineMOD objects.

rather than absolute depth. This achieves 84.9% ADD-2cm accuracy with 13.5mm mean ADD error—more than 2× improvement over RGB-only.

Key findings:

- Geometric translation (predict Z, derive X/Y) significantly reduces learning complexity
- Depth sensor provides strong prior: model learns only $\pm 10\text{--}30\text{mm}$ offset
- Homoscedastic uncertainty weighting eliminates manual loss weight tuning
- Four objects (cat, duck, eggbox, glue) achieve 100% accuracy with RGB-D
- Iron, phone, drill, and benchvise remain challenging due to reflective/texture-less surfaces and large sizes
- Data augmentation (color jitter, bbox jitter) decreased accuracy, highlighting pose estimation’s sensitivity to geometric constraints

References

- [1] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016. 2
- [2] Stefan Hinterstoisser, Vincent Lepetit, Slobodan Ilic, Stefan Holzer, Gary Bradski, Kurt Konolige, and Nassir Navab. Model based training, detection and pose estimation of texture-less 3d objects in heavily cluttered scenes. In *Asian Conference on Computer Vision (ACCV)*, pages 548–562. Springer, 2012. 2, 4
- [3] Glenn Jocher, Ayush Chaurasia, and Jing Qiu. Ultralytics yolov8, 2023. 2
- [4] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7482–7491, 2018. 1, 4
- [5] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. 5
- [6] Chen Wang, Danfei Xu, Yuke Zhu, Roberto Martín-Martín, Cewu Lu, Li Fei-Fei, and Silvio Savarese. Densefusion: 6d object pose estimation by iterative dense fusion. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3343–3352, 2019. 1, 2, 7
- [7] Yu Xiang, Tanner Schmidt, Venkatraman Narayanan, and Dieter Fox. Posecnn: A convolutional neural network for 6d object pose estimation in cluttered scenes. In *Robotics: Science and Systems (RSS)*, 2018. 1, 2, 7
- [8] Danfei Xu, Dragomir Anguelov, and Ashesh Jain. Pointfusion: Deep sensor fusion for 3d bounding box estimation. *arXiv preprint arXiv:1711.10871*, 2018. 7